

Prac1 :

```
#include<iostream>

#include<string>

using namespace std;

#define max 100

class StudentArray{
    private:
        int count;
        string students[max];
    public:
        StudentArray(){count=0;}
        void insert(string name){
            if(count>=max){
                cout<<"Array is Full. Cannot insert more Students.\n";
                return;
            }
            students[count++]=name;
            cout<<"Student inserted successfully.\n";
        }
        void deleteByname(string name){
            int pos =-1;
            for(int i=0; i<count; i++){
                if(students[i]==name){
                    pos=i;
                    break;
                }
            }
            if(pos==-1){
                cout<<"Student not found.\n";
                return;
            }
        }
    }
```

```

        for(int i=pos; i<count-1; i++){
            students[i]=students[i+1];
        }
        count--;
        cout<<"Student deleted successfully.\n";
    }
// void deleteAt(int pos){
//     if(pos<0 || pos>count){
//         cout<<"Invalid position.\n";
//         return;
//     }
//     for(int i=pos; i<count-1;i++){
//         students[i]=students[i+1];
//     }
//     count--;
//     cout<<"Student deleted successfully.\n";
// }
void traverse(){
    if(count==0){
        cout<<"No Students in the array.\n";
        return;
    }
    cout<<"Students Names : \n";
    for(int i=0;i<count;i++){
        cout<<i<<": "<<students[i]<<endl;
    }
}

};

int main(){
    StudentArray sa;

```

```

int choice;

string name;

do{
    cout<<"\n--Student Record--\n";
    cout<<"1. Insert Student.\n";
    cout<<"2. Delete Student.\n";
    cout<<"3. Display Student.\n";
    cout<<"4. Exit.\n";
    cout<<"Enter your choice : ";
    cin>>choice;
    switch(choice) {
        case 1:
            cout<<"Enter Student name : ";
            cin>>name;
            sa.insert(name);
            break;
        case 2: cout<<"Enter Student name to Delete : ";
                cin>>name;
                sa.deleteByname(name);
                break;
        case 3: sa.traverse();
                break;
        case 4: cout<<"Exiting....";
                return 0;
        default : cout<<"Invalid choice!";
    }
}while(choice!=4);

    return 0;

}

```

Prac 2:

```
#include <iostream>

using namespace std;

int main() {
    int r, c, n;

    cout << "Enter number of trains: ";
    cin >> r;

    cout << "Enter number of stations: ";
    cin >> c;

    cout << "Enter number of timings available: ";
    cin >> n;

    int sparse[n + 1][3];

    // Header
    sparse[0][0] = r;
    sparse[0][1] = c;
    sparse[0][2] = n;

    cout << "\nEnter data (Train No, Station No, Time):\n";
    for (int i = 1; i <= n; i++) {
        cin >> sparse[i][0] >> sparse[i][1] >> sparse[i][2];
    }

    cout << "\nSparse Matrix Representation:\n";
    cout << "Train Station Time\n";
    for (int i = 0; i <= n; i++) {
        cout << sparse[i][0] << "    "
            << sparse[i][1] << "    "
            << sparse[i][2] << endl;
    }

    return 0;
}
```

Prac 3:

```
#include <iostream>

#include <string>

using namespace std;

int main() {

    int n;

    string name[10], temp;

    cout << "Enter number of students: ";

    cin >> n;

    for (int i = 0; i < n; i++) {

        cout << "Enter name " << i+1 << ": ";

        cin >> name[i];

    }

    // Bubble Sort

    for (int i = 0; i < n - 1; i++) {

        for (int j = 0; j < n - i - 1; j++) {

            if (name[j] > name[j + 1]) {

                temp = name[j];

                name[j] = name[j + 1];

                name[j + 1] = temp;

            }

        }

    }

    cout << "\nSorted Student Names:\n";

    for (int i = 0; i < n; i++) {

        cout << name[i] << " ";

    }

    return 0;

}
```

Prac 4:

```
#include <iostream>

using namespace std;

struct Student {
    int roll;
    char name[20];
    Student* next;
};

Student* head = NULL;

// Insert at beginning
void insert() {
    Student* temp = new Student();
    cout << "Enter Roll No: ";
    cin >> temp->roll;
    cout << "Enter Name: ";
    cin >> temp->name;
    temp->next = head;
    head = temp;
}

// Display all records
void display() {
    Student* ptr = head;
    if (ptr == NULL) {
        cout << "No records found.\n";
        return;
    }
    cout << "\nStudent Records:\n";
    while (ptr != NULL) {
        cout << ptr->roll << " " << ptr->name << endl;
        ptr = ptr->next;
    }
}
```

```

}

// Search by roll number
void search(int key) {
    Student* ptr = head;
    while (ptr != NULL) {
        if (ptr->roll == key) {
            cout << "Student Found: "
                << ptr->roll << " " << ptr->name << endl;
            return;
        }
        ptr = ptr->next;
    }
    cout << "Student not found.\n";
}

// Delete by roll number
void deleteStudent(int key) {
    Student* ptr = head;
    Student* prev = NULL;
    while (ptr != NULL && ptr->roll != key) {
        prev = ptr;
        ptr = ptr->next;
    }
    if (ptr == NULL) {
        cout << "Student not found.\n";
        return;
    }
    if (prev == NULL) { // delete first node
        head = ptr->next;
    } else {
        prev->next = ptr->next;
    }
}

```

```

delete ptr;

    cout << "Student deleted successfully.\n";
}

int main() {
    int ch, roll;
    do {
        cout<<"\n====Student Record Management====";
        cout << "\n1.Insert\n2.Display\n3.Search\n4.Delete\n5.Exit\n";
        cout << "Enter choice: ";
        cin >> ch;
        switch (ch) {
            case 1: insert();
                    break;
            case 2: display();
                    break;
            case 3: cout << "Enter roll number to search: ";
                    cin >> roll;
                    search(roll);
                    break;
            case 4: cout << "Enter roll number to delete: ";
                    cin >> roll;
                    deleteStudent(roll);
                    break;
            case 5: cout<<"Exiting...." ;
                    return 0;
            default: cout<<"Invalid choice";
        }
    } while (ch != 5);
    return 0;
}

```


Prac 5:

```
#include <iostream>

using namespace std;

struct Process {

    int pid;

    int burst;

    Process* next;

};

Process* head = NULL;

// Insert process

void insert(int id, int bt) {

    Process* temp = new Process();

    temp->pid = id;

    temp->burst = bt;

    if (head == NULL) {

        head = temp;

        temp->next = head;

    } else {

        Process* ptr = head;

        while (ptr->next != head)

            ptr = ptr->next;

        ptr->next = temp;

        temp->next = head;

    }

}

// Round Robin Scheduling

void roundRobin(int tq) {

    Process* ptr = head;

    cout << "\nExecution Order:\n";

    while (true) {

        bool done = true;
```

```

Process* temp = head;

do {
    if (temp->burst > 0) {
        done = false;
        if (temp->burst > tq) {
            cout << "P" << temp->pid << " executes for " << tq << endl;
            temp->burst -= tq;
        } else {
            cout << "P" << temp->pid
                << " executes for " << temp->burst << " (Completed)\n";
            temp->burst = 0;
        }
        temp = temp->next;
    } while (temp != head);
    if (done)
        break; }

}

int main() {
    int n, bt, tq;
    cout << "Enter number of processes: ";
    cin >> n;
    for (int i = 1; i <= n; i++) {
        cout << "Enter burst time of P" << i << ": ";
        cin >> bt;
        insert(i, bt);
    }
    cout << "Enter time quantum: ";
    cin >> tq;
    roundRobin(tq);
    return 0;
}

```

Prac 6:

```
#include <iostream>

#include <stack>

using namespace std;

int main() {

    stack<string> undo, redo;

    string text = "";

    int choice;

    string input;

    do {

        cout << "\n1.Insert Text\n2.Undo\n3.Redo\n4.Display Text\n5.Exit\n";

        cout << "Enter your choice: ";

        cin >> choice;

        cin.ignore();

        switch (choice) {

            case 1: undo.push(text);

                cout << "Enter text to add: ";

                getline(cin, input);

                text += input;

                while (!redo.empty())

                    redo.pop();

                break;

            case 2:

                if (!undo.empty()) {

                    redo.push(text);

                    text = undo.top();

                    undo.pop();

                } else {

                    cout << "Nothing to Undo!\n";

                }

                break;
```

```
case 3:

    if (!redo.empty()) {
        undo.push(text);
        text = redo.top();
        redo.pop();
    } else {
        cout << "Nothing to Redo!\n";
    }

    break;

case 4: cout << "Current Text: " << text << endl;

    break;

case 5: cout << "Exiting...\n";

    break;

default: cout << "Invalid Choice!\n";

    }

} while (choice != 5);


return 0;

}
```

Prac 7:

```
#include <iostream>

using namespace std;

#define SIZE 5

int cq[SIZE], front = -1, rear = -1;

void enqueue(int job) {
    if ((rear + 1) % SIZE == front) {
        cout << "Printer Queue Full\n";
        return;
    }
    if (front == -1) front = 0;
    rear = (rear + 1) % SIZE;
    cq[rear] = job;
}

void dequeue() {
    if (front == -1) {
        cout << "Printer Queue Empty\n";
        return;
    }
    cout << "Printing Job: " << cq[front] << endl;
    if (front == rear)
        front = rear = -1;
    else
        front = (front + 1) % SIZE;
}

int main() {
    enqueue(101);
    enqueue(102);
    enqueue(103);
    dequeue();
    dequeue();
    return 0;
}
```

Prac 8:

```
#include <iostream>

using namespace std;

struct Student {

    int roll;

    string name;

    Student* left, * right;

};

Student* insert(Student* root, int roll, string name) {

    if (root == NULL) {

        Student* temp = new Student();

        temp->roll = roll;

        temp->name = name;

        temp->left = temp->right = NULL;

        return temp;

    }

    if (roll < root->roll)

        root->left = insert(root->left, roll, name);

    else if (roll > root->roll)

        root->right = insert(root->right, roll, name);

    return root;

}

void inorder(Student* root) {

    if (root != NULL) {

        inorder(root->left);

        cout << "Roll: " << root->roll

            << " Name: " << root->name<< endl;

        inorder(root->right);

    }

}

void search(Student* root, int roll) {
```

```

if (root == NULL) {
    cout << "Record Not Found!\n";
    return;
}
if (roll == root->roll) {
    cout << "Record Found!\n";
    cout << "Name: " << root->name<< endl;
}
else if (roll < root->roll)
    search(root->left, roll);
else
    search(root->right, roll);
}

int main() {
    Student* root = NULL;
    int choice, roll;
    string name;
    do {
        cout << "\n1.Insert\n2.Search\n3.Display\n4.Exit\n";
        cout << "Enter choice: ";
        cin >> choice;
        switch (choice) {
            case 1:
                cout << "Enter Roll No: ";
                cin >> roll;
                cout << "Enter Name: ";
                cin >> name;
                root = insert(root, roll, name);
                break;
            case 2:
                cout << "Enter Roll No to Search: ";

```

```
    cin >> roll;

    search(root, roll);

    break;

case 3:

    cout << "Student Records (Sorted):\n";

    inorder(root);

    break;

case 4:

    cout << "Exiting...\n";

    break;

default:

    cout << "Invalid Choice!\n";

}

} while (choice != 4);

return 0;

}
```


Parac 9:

```
#include <iostream>

#include <vector>

using namespace std;

void DFS(int station, vector<bool>& visited, vector<int> adj[]) {

    visited[station] = true;

    cout << "Station " << station << " visited" << endl;

    // Visit all connected stations
    for (int i = 0; i < adj[station].size(); i++) {

        int nextStation = adj[station][i];

        if (!visited[nextStation])

            DFS(nextStation, visited, adj);

    }

}

int main() {

    int n; // Number of stations

    cout << "Enter number of stations: ";

    cin >> n;

    vector<int> adj[n]; // adjacency list

    int m; // Number of connections/routes

    cout << "Enter number of metro connections: ";

    cin >> m;

    cout << "Enter connections (station1 station2):\n";

    for (int i = 0; i < m; i++) {

        int u, v;
```

```
    cin >> u >> v;

    adj[u].push_back(v);

    adj[v].push_back(u); // since metro routes are usually bidirectional
}

vector<bool> visited(n, false);

int start;

cout << "Enter starting station for DFS: ";

cin >> start;

cout << "\nDFS Traversal of Metro Rail System:\n";

DFS(start, visited, adj);

return 0;
}
```

Prac 10:

```
#include <iostream>

#include<vector>

#include<string>

using namespace std;

int main() {

    int n;

    string name;

    vector<string> students(n);


    cout<<"\nEnter the numbers of registered students : ";

    cin>>n;


    cout<<"\nEnter name of registered students:\n ";

    for(int i=0; i<n; i++){

        cin>>students[i];

    }


    cout<<"\nEnter name of student to check registration: ";

    cin>>name;


    for (int i = 0; i < n; i++) {

        if (students[i] == name) {

            cout<<"\nStudent is registered";

            return 0;

        }

    }

    cout << "\nStudent is not registered";

    return 0;

}
```

Prac 11:

```
#include <iostream>

using namespace std;

int main() {
    int n;

    cout << "Enter number of students: ";

    cin >> n;

    int marks[n];

    cout << "Enter marks of students:\n";

    for (int i = 0; i < n; i++) {
        cin >> marks[i];
    }

    // Bubble Sort

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (marks[j] > marks[j + 1]) {
                int temp = marks[j];
                marks[j] = marks[j + 1];
                marks[j + 1] = temp;
            }
        }
    }

    cout << "Sorted marks in ascending order:\n";

    for (int i = 0; i < n; i++) {
        cout << marks[i] << " ";
    }

    cout << endl;

    return 0;
}
```

Prac 12:

```
#include <iostream>

using namespace std;

int main() {

    int n;

    cout << "Enter number of products: ";

    cin >> n;

    float prices[n];

    cout << "Enter prices of products:\n";

    for (int i = 0; i < n; i++) {

        cin >> prices[i];

    } // Selection Sort

    for (int i = 0; i < n - 1; i++) {

        int minIndex = i;

        for (int j = i + 1; j < n; j++) {

            if (prices[j] < prices[minIndex]) {

                minIndex = j;

            }

        }

        if (minIndex != i) {

            float temp = prices[i];

            prices[i] = prices[minIndex];

            prices[minIndex] = temp;

        }

    }

    cout << "Product prices in ascending order:\n";

    for (int i = 0; i < n; i++) {

        cout << prices[i] << " ";

    } cout << endl;

    return 0;

}
```

Prac 13:

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
int main() {
```

```
    int n;
```

```
    cout << "Enter number of students: ";
```

```
    cin >> n;
```

```
    string names[n];
```

```
    cout << "Enter names of students:\n";
```

```
    for (int i = 0; i < n; i++) {
```

```
        cin>>names[i];
```

```
    }
```

```
    // Insertion Sort
```

```
    for (int i = 1; i < n; i++) {
```

```
        string key = names[i];
```

```
        int j = i - 1;
```

```
        while (j >= 0 && names[j] > key) {
```

```
            names[j + 1] = names[j];
```

```
            j--;
```

```
        }
```

```
        names[j + 1] = key;
```

```
    cout << "Student names in ascending order:\n";
```

```
    for (int i = 0; i < n; i++) {
```

```
        cout << names[i] << endl;
```

```
    }
```

```
    return 0;
```

```
}
```